www.Syskriti.in

SQL 200

How Databases Are Designed, Optimized, Deployed, and Kept in Sync

Building on SQL 100 — from querying data to understanding the systems behind it

---

The Big Picture: From One Table to a System

In SQL 100, you learned to think of data as tables and to query them with SELECT. But in a real organization, there are bigger questions: How should we structure those tables? How do we make queries fast on millions of rows? Where does the database physically live? And what happens when multiple copies of the data need to stay in sync?

This guide connects all of those pieces into one logical story. Here's the roadmap:

1) **Normalization & Denormalization** — how to structure your tables.
2) **Indexing & Partitioning** — how to make queries fast.
3) **OLTP vs. Data Warehouse** — two different purposes for databases.
4) **Who Works Where** — how SEs, Data Engineers, and BA/DAs each interact with these systems.
5) **Deployment** — Cloud, On-Prem, and Hybrid.
6) **Replication & Consistency** — keeping copies of data reliable across the globe.

Normalization: Removing Redundancy

Imagine you have one big table where every order repeats the customer's name, phone number, branch, and address. If a customer places 3 orders, that same contact info appears 3 times. This creates two problems: wasted storage, and the risk that one copy gets updated while the others don't (an update anomaly).

Normalization is the process of splitting one large table into smaller, related tables so that each piece of information is stored exactly once.

Before: One Flat Table (Denormalized)

OrderID	CustName	Phone	Branch	Product	Qty	Address
1001	Ali	555-01	Dallas	Laptop	1	123 Main St
1002	Ali	555-01	Dallas	Mouse	2	123 Main St

@copyright: Syskriti Pvt Ltd 2026.

1003	Ali	555-01	Dallas	Monitor	1	123 Main St

> **Problem:** *Ali's name, phone, branch, and address are duplicated in every row. If Ali moves, you'd have to update 3 rows — miss one and the data is inconsistent.*

After: Normalized into Two Tables

Customers Table (each customer stored once)

CustID	Name	Phone	Branch	Address
A1	Ali	555-01	Dallas	123 Main St

Orders Table (references the customer by ID)

OrderID	CustID	Product	Qty	Date
1001	**A1**	Laptop	1	2024-01-22
1002	**A1**	Mouse	2	2024-01-23
1003	**A1**	Monitor	1	2024-01-24

> **Key insight:** *The CustID column (A1) in the Orders table is a foreign key that points back to the Customers table. Ali's info is stored once; orders just reference it. This is the Join Table pattern — you use JOIN in SQL to reconnect them when needed.*

Levels of Normalization (Quick Overview)

Normalization has formal levels (called "normal forms"). For now, the key idea is:

- **1NF:** Every cell has a single value (no lists stuffed into one cell).
- **2NF:** Every non-key column depends on the entire primary key, not just part of it.
- **3NF:** No non-key column depends on another non-key column (eliminate transitive dependencies).

Most production OLTP databases aim for 3NF. Software Engineers and Database Engineers are typically the ones who design and enforce this normalized structure.

@copyright: Syskriti Pvt Ltd 2026.

Denormalization: Trading Structure for Speed

If normalization removes duplication, denormalization deliberately adds some back. Why would you do that?

Because JOINs are expensive. When a Data Analyst needs to run a report combining customers, orders, products, and suppliers, a fully normalized database might require joining 4–5 tables together. On millions of rows, that can be slow.

Denormalization pre-joins or pre-aggregates data so that common queries run faster, at the cost of storing some information more than once.

Star Schema & Snowflake Schema

Denormalization in a Data Warehouse doesn't mean dumping everything into one giant flat table. Warehouses use structured patterns:

- **Star Schema:** One central fact table (e.g., Sales Transactions) surrounded by dimension tables (Customer, Product, Date, Store). The fact table holds the measurable events; the dimension tables describe the "who, what, when, where." Analysts join fact to dimension — typically just one hop — which is much faster than the multi-hop JOINs of a fully normalized OLTP system.
- **Snowflake Schema:** Similar to a star, but the dimension tables are themselves partially normalized (e.g., a Product dimension links to a separate Category table). This saves some storage but adds JOINs back, so it sits between full normalization and a pure star.

> **Why it matters:** *When someone says a warehouse is "denormalized," they almost always mean a star or snowflake schema — not a single enormous table. Understanding this prevents a common misconception.*

	Normalization	Denormalization
Goal	Eliminate redundancy	Speed up reads/queries
Duplicates?	No (or minimal)	Yes, intentionally
JOINs needed?	Many (multi-hop)	Few (one-hop in star schema)
Write speed	Fast (small, focused tables)	Slower (more data to update)
Read speed	Slower (needs multi-hop JOINs)	Fast (pre-combined)
Schema pattern	3NF (third normal form)	Star or Snowflake schema

@copyright: Syskriti Pvt Ltd 2026.

Used in	OLTP systems	Data Warehouses

> *This is the fundamental trade-off in database design: normalize for clean writes, denormalize for fast reads. This distinction leads directly to the split between OLTP and Data Warehouses.*

Indexing: The Speed Trick for Columns

Think of a textbook. If you want to find every mention of “ACID,” you could read every page from start to finish. Or you could look it up in the index at the back, which tells you exactly which pages to go to.

A database index works the same way. It’s a separate data structure (usually a B-tree) that the database builds on one or more columns, allowing it to jump directly to matching rows instead of scanning the entire table.

How It Helps

Without an index: the database performs a full table scan — checking every row. On 10 million rows, this is slow.

With an index on that column: the database looks up the index first, finds the exact row locations, and reads only those rows.

```
CREATE INDEX idx_customer_name ON Customers(Name);
```

The Trade-Off

- **Faster reads:** SELECT queries that filter or sort by the indexed column are dramatically faster.
- **Slower writes:** Every INSERT, UPDATE, or DELETE must also update the index. More indexes mean more overhead on writes.
- **Extra storage:** Each index takes up disk space.

> **Rule of thumb:** *Index columns you frequently search, filter, or join on. Don’t index everything — that would slow down every write operation.*

*Notice the pattern: indexing is a **column-level** optimization. You’re speeding up access to specific fields. This contrasts with partitioning, which works at the row level.*

@copyright: Syskriti Pvt Ltd 2026.

Partitioning: Splitting the Table by Rows

When a table grows to hundreds of millions of rows, even good indexes can't prevent slowdowns. Partitioning solves this by physically splitting the table into smaller chunks based on row values.

Think of it like filing cabinets. Instead of one enormous cabinet for all orders, you have one cabinet per year: Orders_2022, Orders_2023, Orders_2024. When a query asks for "orders in 2024," the database only opens one cabinet instead of searching through all of them. This is called partition pruning.

Common Partitioning Strategies

- **Range partitioning:** Split by date ranges (e.g., one partition per month or year). Most common.
- **List partitioning:** Split by category (e.g., one partition per region: USA, Europe, India).
- **Hash partitioning:** Split by a computed hash value, distributing rows evenly. Useful when no natural range exists.

Indexing vs. Partitioning: How They Relate

	Indexing	Partitioning
Operates on	Columns	Rows
What it does	Builds a lookup map for fast column-based search	Physically splits the table into smaller chunks
Analogy	Book index (find the right page)	Filing cabinets (open the right drawer)
Used together?	Yes — you can index within each partition	Yes — partitions narrow the scope, indexes speed up the search within

> *In practice, large production systems use BOTH: partitioning narrows down which chunk of data to scan, and indexing speeds up the search within that chunk.*

OLTP vs. Data Warehouse: Two Different Purposes

Not all databases serve the same goal. In most organizations, data lives in (at least) two kinds of systems:

@copyright: Syskriti Pvt Ltd 2026.

OLTP — Online Transaction Processing

This is the production database that powers the live application. Every time a customer places an order, a user signs up, or an employee updates a record, it's an OLTP system doing the work.

- **Optimized for writes:** fast inserts, updates, and deletes.
- **Normalized (3NF):** tables are split to avoid redundancy and ensure data integrity.
- **Many small transactions:** thousands of concurrent users each touching a few rows at a time.
- **Who builds and maintains it:** Software Engineers (SE) and Database Engineers.

Data Warehouse

This is a separate system built specifically for analysis and reporting. Data is extracted from the OLTP system and loaded into the warehouse, often through an ETL or ELT process.

- **Optimized for reads:** fast complex queries across large datasets.
- **Denormalized (Star/Snowflake schema):** tables are pre-joined to minimize JOINs at query time.
- **Few heavy queries:** analysts running reports on millions or billions of rows.
- **Read-only on production data:** analysts SELECT from production tables but don't modify them. They may create temp tables, views, or write to sandbox schemas for their own analytical work.
- **Who uses it:** Business Analysts (BA) and Data Analysts (DA).

ETL vs. ELT: How Data Moves Between Systems

Data doesn't magically appear in the warehouse. It flows there through a pipeline:

- **ETL (Extract, Transform, Load):** The classic approach. Data is extracted from OLTP, transformed (cleaned, joined, aggregated) in a staging area, then loaded into the warehouse. The transformation happens before the data reaches the warehouse.
- **ELT (Extract, Load, Transform):** The modern approach, popular with cloud warehouses like Snowflake and BigQuery. Raw data is extracted and loaded directly into the warehouse first, then transformed inside the warehouse using SQL. This leverages the warehouse's powerful compute engine for the heavy lifting.

> **Why ELT is gaining ground:** *Cloud warehouses can handle massive transformations with on-demand compute power. Loading raw data first also means you preserve the original data and can re-transform it later if business rules change — you don't lose anything in translation.*

	OLTP	Data Warehouse

@copyright: Syskriti Pvt Ltd 2026.

Purpose	Run the business (transactions)	Analyze the business (reporting)
Optimized for	Writes (INSERT/UPDATE/DELETE)	Reads (SELECT)
Structure	Normalized (3NF)	Denormalized (Star/Snowflake)
Users	SE / Engineers / Applications	BA / DA / Analysts
Query type	Many small, fast queries	Few large, complex queries
Data freshness	Real-time (live)	Periodic (ETL/ELT schedule)
Access	Read + Write	Read-only on production; may write to sandbox
Pipeline	Source of data	Receives data via ETL or ELT

> **The connection:** *Normalization feeds OLTP (clean writes by Engineers). Denormalization feeds the Warehouse (fast reads by Analysts). Data Engineers build the ETL/ELT pipelines that bridge the two. Indexing and partitioning are used in BOTH systems, but are especially critical in the Warehouse where queries scan massive datasets.*

Who Works Where: The Three-Lane Role Map

Understanding which roles interact with which system clears up a lot of confusion. In practice, there are three lanes, not two:

Lane 1: Software Engineers / Database Engineers → OLTP

- Design the normalized schema (tables, constraints, relationships).
- Write application code that performs INSERT, UPDATE, DELETE operations.
- Optimize write performance with appropriate indexes.
- Ensure ACID compliance for data integrity.

Lane 2: Data Engineers → The Bridge (ETL/ELT Pipelines)

- Build and maintain the pipelines that extract data from OLTP and load it into the Warehouse.
- Design the transformation logic: cleaning, joining, aggregating, deduplication.
- Monitor pipeline health, handle failures, and manage scheduling (hourly, nightly, near-real-time).
- Design the warehouse schema (star/snowflake) in collaboration with analysts.
- Often the ones who implement partitioning and indexing strategies in the warehouse.

Lane 3: Business Analysts / Data Analysts → Data Warehouse

@copyright: Syskriti Pvt Ltd 2026.

- Write SELECT queries (DQL) to generate reports, dashboards, and insights.
- Work with denormalized, pre-joined tables designed for fast reads.
- Read-only on production warehouse tables — don't modify source data.
- May create temp tables, views, materialized views, or write to sandbox/staging schemas for their own analysis.
- Request new tables, columns, or aggregations from the Data Engineering team.

SE / DB Engineers	Data Engineers	BA / DA
Build & maintain OLTP	Build ETL/ELT pipelines	Query the Warehouse
Normalized schema (3NF)	Transform & move data	Star/Snowflake schema
INSERT / UPDATE / DELETE	Schedule, monitor, repair	SELECT (read-only on prod)
Optimize writes + ACID	Design warehouse schema	Reports, dashboards, insights

> **Why this matters:** *When a BA says "the report is slow," the fix usually involves the Data Engineer adjusting partitioning, indexing, or denormalization of the warehouse tables. When an SE says "writes are slow," the fix is in the OLTP schema design or indexing strategy. When data is stale or missing, the Data Engineer investigates the ETL/ELT pipeline.*

Deployment: Where Does the Database Live?

Every database runs on physical hardware somewhere. There are three deployment models:

On-Premises ("On-Prem")

The database runs on servers that your organization owns and manages in its own data center. You control everything: hardware, security, backups, and network. This gives maximum control but requires significant infrastructure investment.

Cloud

The database runs on infrastructure managed by a cloud provider (AWS, Azure, Google Cloud). The provider handles hardware, scaling, backups, and availability. You access it over the internet. This gives flexibility and reduces operational burden, but means trusting a third party with your data.

Hybrid

A mix of both: some databases or workloads stay on-prem (often for regulatory or latency reasons), while others run in the cloud. Data syncs between them. Many large enterprises use this model during cloud migration or for compliance.

@copyright: Syskriti Pvt Ltd 2026.

> *The deployment model doesn't change how SQL works — your queries are the same. But it affects latency, cost, scalability, and how replication is handled, which we'll cover next.*

Replication: Keeping Copies in Sync

When your business operates across multiple regions, you often need copies of the same database in different locations. This is replication: maintaining multiple copies of data across different servers or geographies.

Why Replicate?

1) **Lower latency:** A customer in India shouldn't wait for a query to travel to a server in the USA and back. A local replica serves data faster.
2) **High availability:** If the USA server goes down, the Europe or India replica can take over (failover).
3) **Load balancing:** Read traffic can be spread across replicas instead of overwhelming one server.

Single-Primary Replication (Most Common)

One server is the primary (accepts reads and writes). The others are replicas (read-only copies that receive updates from the primary).

	USA (D1)	Europe (D2)	India (D3)
Customer Table	Full copy	Full copy	Full copy
Role	Primary (read + write)	Replica (read-only)	Replica (read-only)

D1 (USA) is the primary: all writes happen here. D2 (Europe) and D3 (India) are replicas that receive updates from D1 and serve local read traffic.

Multi-Primary Replication (Less Common, More Complex)

In some systems, multiple servers can all accept writes. D1, D2, and D3 would each handle both reads and writes for their local region.

- **Advantage:** No single bottleneck for writes. Each region is self-sufficient.
- **Challenge:** Conflict resolution. What happens if a user in the USA and a user in India update the same customer's phone number at the same time? The system needs rules to decide which write wins (e.g., "last write wins," or application-level conflict logic).

@copyright: Syskriti Pvt Ltd 2026.

> **In practice:** *Most systems start with single-primary replication because it's simpler and avoids conflict resolution entirely. Multi-primary is used when write latency across regions is unacceptable and the team can handle the added complexity.*

The Consistency Problem

Here's the fundamental tension: when a customer in the USA updates their phone number in D1, how quickly do D2 and D3 reflect that change?

Synchronous Replication

The write isn't confirmed until ALL replicas have the update. This guarantees strong consistency — every replica always has the same data — but it's slow, because the write must wait for confirmation from Europe and India before it can complete.

- **Pros:** Perfect consistency. No stale reads from any replica.
- **Cons:** High latency on writes. If one replica is unreachable, all writes stall.

Asynchronous Replication

The write is confirmed as soon as D1 records it. D2 and D3 receive the update shortly after (milliseconds to seconds). This is fast, but there's a window where the replicas are stale.

- **Pros:** Fast writes. No single replica can block the system.
- **Cons:** Temporary inconsistency. A user in India might see the old phone number for a few seconds.

Eventual Consistency

The formal term for this pattern is **eventual consistency**: if no new writes occur, all replicas will *eventually* converge to the same state. The system doesn't guarantee that you'll read the latest data at any given moment — only that the data will catch up.

This is one of the most important concepts in distributed systems. You'll encounter it in every discussion about replicated databases, distributed caches, and cloud architectures.

> **The Balance:** *Every replicated system must choose where to sit on the spectrum between strong consistency (slow, safe) and eventual consistency (fast, temporarily stale). Most real-world systems use asynchronous replication and accept small windows of inconsistency, because the latency cost of synchronous replication is too high for global systems. The choice depends on what's at stake: a banking transaction might demand strong consistency, while a social media feed is fine with eventual consistency.*

@copyright: Syskriti Pvt Ltd 2026.

How Everything Connects

Let's tie every concept together into one coherent picture:

1) **Engineers design normalized tables** (3NF) for the OLTP system. This minimizes redundancy and keeps writes fast and safe (ACID).
2) **The OLTP system is indexed** on columns that the application queries frequently (e.g., customer ID, order date).
3) **Data Engineers build ETL/ELT pipelines** that extract data from OLTP, transform it, and load it into the Data Warehouse.
4) **In the Warehouse, data is denormalized** into star or snowflake schemas — wide, pre-joined tables optimized for analytical queries.
5) **The Warehouse is partitioned** (e.g., by date or region) and indexed so analysts can run heavy queries on massive datasets efficiently.
6) **Analysts query the Warehouse** with SELECT (read-only on production data), generating reports and dashboards. They may write to sandbox schemas for ad-hoc analysis.
7) **Both systems can be deployed** on-prem, in the cloud, or in a hybrid model depending on cost, compliance, and latency needs.
8) **If the business is global,** the database is replicated across regions. The team must balance strong consistency vs. eventual consistency depending on what the data is used for.

> **The thread that runs through everything:** *every design decision — normalize vs. denormalize, add an index or not, partition by date or region, replicate synchronously or asynchronously, ETL vs. ELT — is a trade-off. Understanding what you're trading and why is what separates someone who writes SQL from someone who understands databases.*

Quick Recap

- **Normalization:** Split tables to store each fact once. Prevents duplication and update anomalies. Used in OLTP (3NF).
- **Denormalization:** Pre-join tables into star or snowflake schemas. Accepts some duplication for faster reads. Used in Warehouses.
- **Indexing:** Column-level optimization. Builds a lookup structure for fast searches. Faster reads, slower writes.
- **Partitioning:** Row-level optimization. Physically splits the table into chunks. Reduces scan scope on large tables.

@copyright: Syskriti Pvt Ltd 2026.

- **OLTP:** The live, transactional database. Normalized. Write-heavy. Managed by Software Engineers.
- **Data Warehouse:** The analytical database. Denormalized. Read-heavy. Used by BA/DA (read-only on production; may write to sandbox).
- **Data Engineers:** Build the ETL/ELT pipelines that bridge OLTP and Warehouse. Design warehouse schemas and optimization strategies.
- **ETL vs. ELT:** ETL transforms before loading; ELT loads raw data first and transforms inside the warehouse.
- **Deployment:** Cloud, On-Prem, or Hybrid. Doesn't change SQL, but affects latency, cost, and replication.
- **Replication:** Copying data across regions for speed and availability. Single-primary (common) or multi-primary (complex).
- **Consistency:** Strong consistency (synchronous, slow) vs. eventual consistency (asynchronous, fast). Most systems choose eventual.

Every concept above is a trade-off. Master the trade-offs, and you'll understand not just how to write SQL, but why databases are designed the way they are.

@copyright: Syskriti Pvt Ltd 2026.